

**UNIVERSIDADE FEDERAL DE
UBERLÂNDIA**

PPGEELT – Pós-Graduação em Engenharia
Elétrica

LUCAS ALBINO MARTINS
12622EEL006

**TRABALHO 08 - RECONHECIMENTO DE FLORES
IRIS USANDO MLP**

Disciplina: EL056 - Redes Neurais Artificiais

Uberlândia

2026

**UNIVERSIDADE FEDERAL DE
UBERLÂNDIA**

PPGEELT – Pós-Graduação em Engenharia
Elétrica

LUCAS ALBINO MARTINS
12622EEL006

**TRABALHO 08 - RECONHECIMENTO DE FLORES
IRIS USANDO MLP**

Disciplina: EL056 - Redes Neurais Artificiais

Trabalho apresentado ao Programa de Pós-Graduação em Engenharia Elétrica (PPGEELT) da Universidade Federal de Uberlândia como requisito parcial de avaliação da disciplina.

Disciplina: EL056 - Redes Neurais Artificiais.

Professor: Prof. Dr. Keiji Yamanaka.

Uberlândia

2026

RESUMO

Este trabalho apresenta o projeto, a implementação e a avaliação de uma rede Perceptron Multicamadas (MLP) para a classificação das três espécies do conjunto de dados Iris (Fisher, 1936) a partir de quatro medidas morfológicas. A rede foi implementada do zero em NumPy, com função Softmax numericamente estável na camada de saída, função de custo entropia cruzada categorial e retropropagação manual, cujos gradientes foram validados por diferenças finitas (erro relativo máximo de $3,5 \times 10^{-8}$ para $\tanh$). Adotou-se padronização z-score ajustada apenas no treino, divisão estratificada 70/15/15, gradiente descendente em mini-batch com momentum e parada antecipada. A arquitetura 4–8–3 ($\tanh$) obteve acurácia de 100,00% no conjunto de teste e de $97,33\% \pm 3,65\%$ na validação cruzada estratificada com cinco partições, desempenho compatível com a literatura e comparável aos baselines de regressão logística multinomial e k -NN. A *Setosa* é reconhecida sem erros, e as confusões restantes concentram-se no par *Versicolor–Virginica*, que não é linearmente separável.

Palavras-chave: Redes neurais artificiais. Perceptron multicamadas. Softmax. Entropia cruzada. Retropropagação. Conjunto Iris.

Sumário

1	Introdução	4
2	Fundamentação Teórica	4
2.1	Perceptron Multicamadas e propagação direta	4
2.2	Função Softmax estável	5
2.3	Entropia cruzada categorial	5
2.4	Gradiente conjunto Softmax + entropia cruzada	5
2.5	Retropropagação e regra de atualização	5
2.6	Inicialização dos pesos	6
3	Base de Dados	6
3.1	Correção das amostras 35 e 38	6
3.2	Análise exploratória	6
4	Metodologia	8
4.1	Pré-processamento e divisão dos dados	8
4.2	Arquitetura e hiperparâmetros	9
4.3	Algoritmo de treinamento	9
4.4	Verificação do gradiente	10
4.5	Experimentos e baselines	10
4.6	Métricas	11
5	Resultados	11
5.1	Verificação do gradiente	11
5.2	Treinamento	11
5.3	Desempenho no conjunto de teste	11
5.4	Validação cruzada e experimentos	12
5.5	Comparação com baselines	13
5.6	Fronteira de decisão	14
6	Discussão	14
7	Conclusão	15
	Referências	16
A	Trechos do código	16

Classificação do conjunto Iris com Perceptron Multicamadas, Softmax e Entropia Cruzada

1 Introdução

O conjunto de dados Iris, publicado por Fisher (1936) para ilustrar a análise discriminante linear, reúne 150 flores de três espécies do gênero *Iris* — *I. setosa*, *I. versicolor* e *I. virginica*, com 50 exemplares cada — descritas por quatro medidas em centímetros: comprimento e largura da sépala e da pétala. Ele é um dos *benchmarks* mais citados em reconhecimento de padrões (DUDA; HART, 1973; DASARATHY, 1980; GATES, 1972) porque reúne, em escala reduzida, uma classe linearmente separável e um par de classes que não é.

Este trabalho atende ao enunciado do Trabalho 08 da disciplina EL056: treinar uma MLP para classificar as flores, com função de ativação Softmax na saída e função de custo entropia cruzada. Para além do mínimo exigido, buscou-se rigor metodológico: implementação sem frameworks de *deep learning*, verificação numérica dos gradientes, ausência de vazamento de dados no pré-processamento, avaliação por validação cruzada estratificada, estudo de sensibilidade a hiperparâmetros e comparação com baselines.

O relatório está organizado da seguinte forma. A Seção 2 apresenta a fundamentação teórica; a Seção 3 descreve a base de dados; a Seção 4, a metodologia; as Seções 5 e 6 apresentam e discutem os resultados; e a Seção 7 conclui o trabalho.

2 Fundamentação Teórica

2.1 Perceptron Multicamadas e propagação direta

Uma MLP com L camadas de pesos mapeia uma entrada $\mathbf{x} \in \mathbb{R}^d$ em um vetor de probabilidades $\hat{\mathbf{y}} \in \mathbb{R}^K$. Em notação matricial por lote (cada linha de $\mathbf{A}^{(0)} = \mathbf{X} \in \mathbb{R}^{N \times d}$ é uma amostra):

$$\mathbf{Z}^{(l)} = \mathbf{A}^{(l-1)}\mathbf{W}^{(l)} + \mathbf{1}\mathbf{b}^{(l)\top}, \quad l = 1, \dots, L, \quad (1)$$

$$\mathbf{A}^{(l)} = f(\mathbf{Z}^{(l)}), \quad l = 1, \dots, L - 1, \quad (2)$$

$$\hat{\mathbf{Y}} = \text{softmax}(\mathbf{Z}^{(L)}), \quad (3)$$

em que $\mathbf{W}^{(l)} \in \mathbb{R}^{n_{l-1} \times n_l}$ e f é a ativação das camadas ocultas:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \quad \text{ReLU}(z) = \max(0, z), \quad (4)$$

com derivadas $\sigma'(z) = \sigma(z)[1 - \sigma(z)]$, $\tanh'(z) = 1 - \tanh^2(z)$ e $\text{ReLU}'(z) = \mathbb{I}[z > 0]$ (HAYKIN, 2001).

2.2 Função Softmax estável

A Softmax converte os *logits* $\mathbf{z} \in \mathbb{R}^K$ em uma distribuição de probabilidade:

$$\hat{y}_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}} = \frac{e^{z_k - m}}{\sum_{j=1}^K e^{z_j - m}}, \quad m = \max_j z_j. \quad (5)$$

A segunda forma é matematicamente idêntica à primeira, pois o fator e^{-m} se cancela no numerador e no denominador. Numericamente, porém, ela garante $z_k - m \leq 0$, de modo que $e^{z_k - m} \in (0, 1]$: não há *overflow* e pelo menos um termo do denominador vale 1.

2.3 Entropia cruzada categorial

Com rótulos codificados em *one-hot* ($y_{nk} = 1$ se a amostra n pertence à classe k), a função de custo média é

$$L = -\frac{1}{N} \sum_{n=1}^N \sum_{k=1}^K y_{nk} \ln \hat{y}_{nk}, \quad (6)$$

que corresponde à log-verossimilhança negativa de um modelo multinomial (GOODFELLOW; BENGIO; COURVILLE, 2016). Na implementação, $\hat{y}$ é limitado ao intervalo $[\varepsilon, 1]$, com $\varepsilon = 10^{-12}$, para evitar $\ln 0$.

2.4 Gradiente conjunto Softmax + entropia cruzada

O jacobiano da Softmax é $\partial \hat{y}_k / \partial z_j = \hat{y}_k(\delta_{kj} - \hat{y}_j)$. Para uma amostra:

$$\frac{\partial L_n}{\partial z_j} = -\sum_k \frac{y_k}{\hat{y}_k} \hat{y}_k(\delta_{kj} - \hat{y}_j) = -y_j + \hat{y}_j \sum_k y_k = \hat{y}_j - y_j, \quad (7)$$

pois $\sum_k y_k = 1$. Para o custo médio no lote, $\boldsymbol{\delta}^{(L)} = (\hat{\mathbf{Y}} - \mathbf{Y})/N$. Esse resultado elimina o cálculo explícito do jacobiano, é numericamente estável e tem gradiente limitado em $[-1, 1]$, o que evita o problema de saturação do erro quadrático com saídas sigmoidais.

2.5 Retropropagação e regra de atualização

Aplicando a regra da cadeia de trás para frente (RUMELHART; HINTON; WILLIAMS, 1986; HAYKIN, 2001):

$$\nabla_{\mathbf{W}^{(l)}} L = \mathbf{A}^{(l-1)\top} \boldsymbol{\delta}^{(l)}, \quad \nabla_{\mathbf{b}^{(l)}} L = \boldsymbol{\delta}^{(l)\top} \mathbf{1}, \quad (8)$$

$$\boldsymbol{\delta}^{(l-1)} = (\boldsymbol{\delta}^{(l)} \mathbf{W}^{(l)\top}) \odot f'(\mathbf{Z}^{(l-1)}). \quad (9)$$

Os parâmetros θ são atualizados por gradiente descendente em mini-batch com momentum:

$$\mathbf{v} \leftarrow \mu \mathbf{v} - \eta \nabla_{\theta} L, \quad \theta \leftarrow \theta + \mathbf{v}, \quad (10)$$

em que η é a taxa de aprendizado e μ o coeficiente de momentum; com $\mu = 0$ recupera-se o gradiente descendente puro.

2.6 Inicialização dos pesos

Para manter a variância das ativações e dos gradientes aproximadamente constante entre camadas, adotou-se a inicialização de Xavier/Glorot (Glorot; BENGIO, 2010), $W \sim \mathcal{N}(0, 2/(n_{in} + n_{out}))$, para *tanh*, sigmoide e a camada Softmax, e a de He (HE *et al.*, 2015), $W \sim \mathcal{N}(0, 2/n_{in})$, para camadas ReLU. Os vieses começam em zero.

3 Base de Dados

A base foi fornecida como `iris_data.xlsx` (150 linhas $\times$ 5 colunas, sem cabeçalho), com documentação no arquivo `iris_names.txt` do repositório UCI (FISHER, 1988). As colunas 1–4 são os atributos em cm (*sepal length*, *sepal width*, *petal length*, *petal width*), armazenados como texto e convertidos para ponto flutuante; a coluna 5 contém o rótulo. Não há valores ausentes, e as classes são balanceadas (33,3% cada). As linhas estão ordenadas por classe em blocos de 50, o que torna obrigatório embaralhar os dados antes de dividi-los.

3.1 Correção das amostras 35 e 38

A documentação registra que duas amostras de *Setosa* divergem do artigo original. Na planilha, ambas valem (4,9; 3,1; 1,5; 0,1); os valores de Fisher são (4,9; 3,1; 1,5; **0,2**) para a amostra 35 e (4,9; **3,6**; **1,4**; 0,1) para a 38. Optou-se por **corrigi-las**, por três razões: os valores verdadeiros são conhecidos e documentados pela própria fonte, de modo que não se trata de imputação; as duas linhas são idênticas na planilha, uma duplicata espúria que daria peso dobrado a um único ponto; e a correção restaura a fidelidade aos dados originais. A decisão é parametrizável (`-corrigir-uci / CORRIGIR_UCI`) e tem efeito desprezível sobre o desempenho, pois ambas as amostras pertencem à classe separável.

3.2 Análise exploratória

A Tabela 1 compara as estatísticas calculadas (após a correção) com as documentadas. Sem a correção, os valores reproduzem exatamente a documentação; com ela, apenas *sepal width* se altera de forma perceptível, por ser o atributo modificado na amostra 38.

Tabela 1: Estatísticas dos atributos (cm), desvio-padrão amostral e correlação de Pearson com o rótulo (0, 1, 2).

Atributo	Mín.	Máx.	Média	DP	Corr. (calc.)	Corr. (UCI)
sepal length	4,3	7,9	5,843	0,828	0,7826	0,7826
sepal width	2,0	4,4	3,057	0,436	-0,4267	-0,4194
petal length	1,0	6,9	3,758	1,765	0,9490	0,9490
petal width	0,1	2,5	1,199	0,762	0,9565	0,9565

As Figuras 1 e 2 confirmam o que a documentação aponta: as medidas de pétala são as mais discriminantes, com correlação com a classe de cerca de 0,95. No plano das pétalas, a *Setosa* forma um aglomerado isolado, enquanto *Versicolor* e *Virginica* se sobrepõem parcialmente. A Figura 3 mostra ainda a forte colinearidade entre *petal length*, *petal width* e *sepal length*.

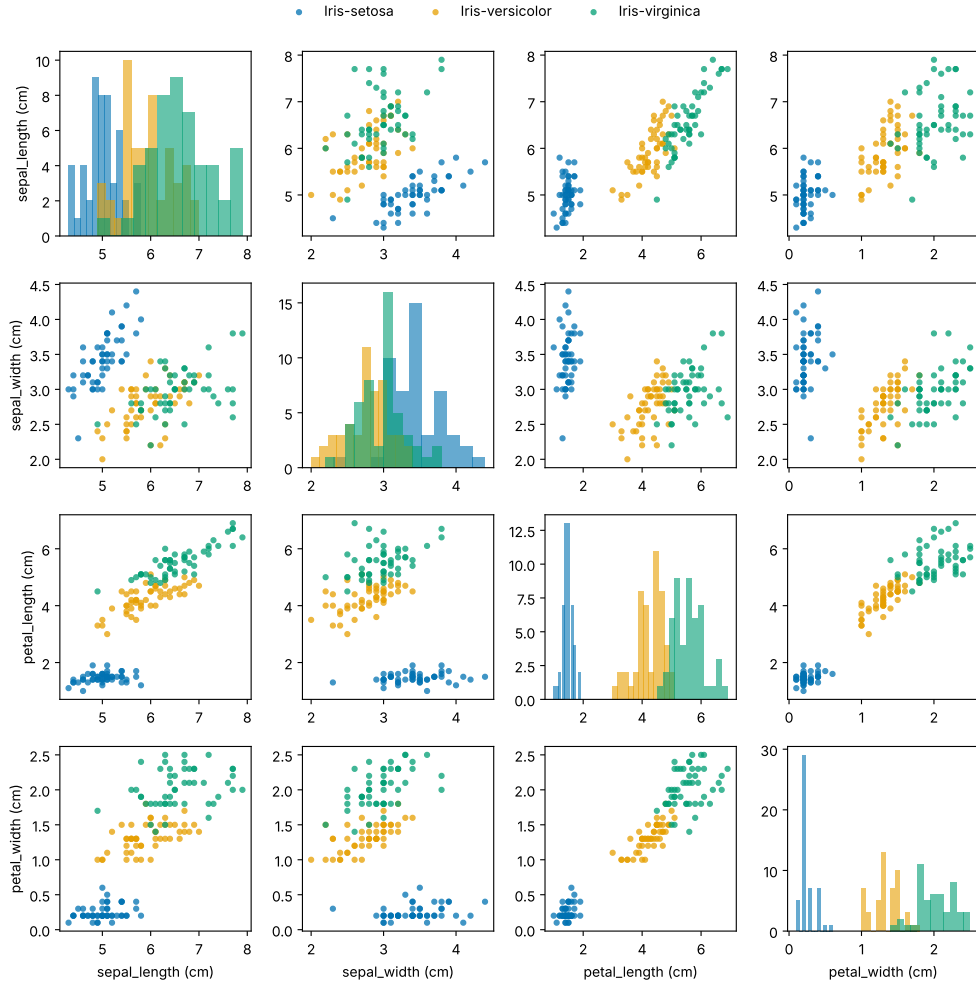


Figura 1: Matriz de dispersão dos quatro atributos, colorida por classe (diagonal: histogramas).

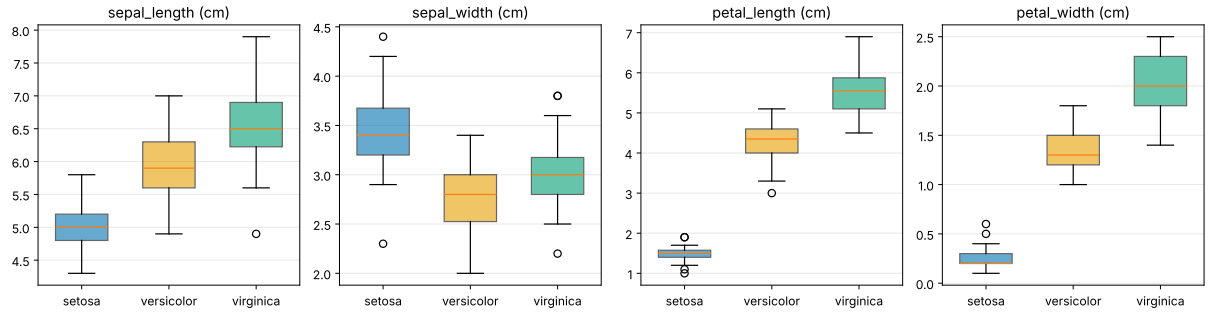


Figura 2: Distribuição de cada atributo por classe.

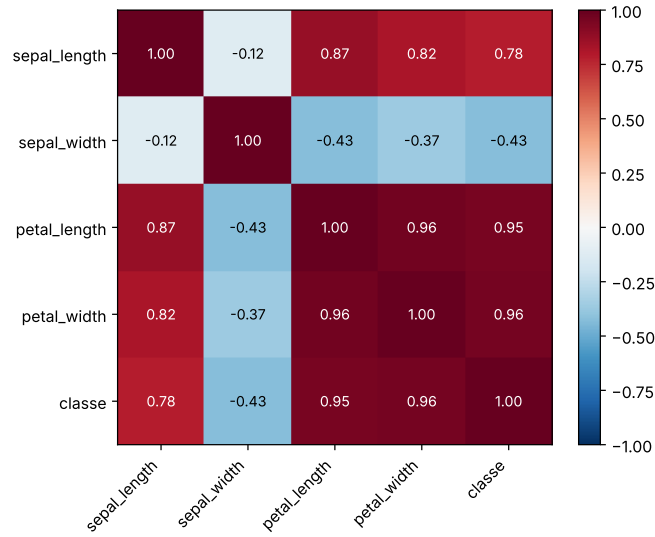


Figura 3: Correlação de Pearson entre atributos e rótulo.

4 Metodologia

4.1 Pré-processamento e divisão dos dados

Os rótulos foram codificados em *one-hot*. Para o holdout, adotou-se uma divisão estratificada 70/15/15 com semente fixa ($SEED = 42$). O arredondamento por classe resulta em 35/8/7 amostras por espécie, ou 105/24/21 no total. O conjunto de validação é usado exclusivamente para a parada antecipada, e o de teste, apenas na avaliação final. Os atributos foram padronizados por z-score, $x' = (x - \mu_{\text{treino}}) / \sigma_{\text{treino}}$, com média e desvio estimados **somente no treino** e aplicados à validação e ao teste, evitando vazamento de informação.

Como 21 amostras de teste produzem uma estimativa de alta variância (cada erro vale cerca de 4,8 pontos percentuais), o desempenho também foi avaliado por validação cruzada **k-fold estratificada** com $k = 5$. Em cada partição, 15% da parte de treino é separada, de forma estratificada, para a parada antecipada, e o z-score é reajustado sobre os 85% restantes.

4.2 Arquitetura e hiperparâmetros

A Tabela 2 resume a configuração principal. A escolha de **uma camada oculta** apoia-se no teorema da aproximação universal e na simplicidade do problema, cuja única fronteira difícil (*Versicolor/Virginica*) é quase linear. **Oito neurônios** dão folga sobre o mínimo necessário sem super-parametrizar as 105 amostras de treino; essa hipótese é testada na Seção 4.5. A **tangente hiperbólica** foi escolhida por ser centrada em zero, o que produz gradientes mais bem condicionados que os da sigmoide, e por ser compatível com a inicialização de Xavier. O mini-batch de 16 amostras resulta em cerca de 7 atualizações por época, e o ruído estocástico ajuda a escapar de platôs. O momentum de 0,9 acelera a descida em direções de curvatura baixa.

Tabela 2: Configuração principal da MLP.

Item	Valor
Arquitetura	4 – 8 (<i>tanh</i>) – 3 (Softmax), 67 parâmetros
Inicialização	Xavier/Glorot (vieses nulos)
Custo	Entropia cruzada categorial, Eq. (6)
Otimizador	SGD em mini-batch, $\eta = 0,05$, $\mu = 0,9$, lote = 16
Parada antecipada	até 2000 épocas, paciência = 100 (perda de validação)
Semente	42

4.3 Algoritmo de treinamento

O Algoritmo 1 sintetiza o procedimento implementado na função **treinar**.

Algoritmo 1 Treinamento da MLP com mini-batch, momentum e parada antecipada

Entrada: $(\mathbf{X}_{tr}, \mathbf{Y}_{tr})$, $(\mathbf{X}_{va}, \mathbf{Y}_{va})$, η , μ , lote B , $E_{\max}$, paciência P

Saída: pesos θ^* da melhor época

```
1: Inicializar  $\mathbf{W}^{(l)}$  (Xavier/He),  $\mathbf{b}^{(l)} \leftarrow \mathbf{0}$ ,  $\mathbf{v} \leftarrow \mathbf{0}$ 
2:  $L^* \leftarrow \infty$ ;  $p \leftarrow 0$ 
3: for  $e = 1$  até  $E_{\max}$  do
4:   Embaralhar índices de treino
5:   for cada lote  $\mathcal{B}$  de tamanho  $B$  do
6:      $\hat{\mathbf{Y}}_{\mathcal{B}} \leftarrow \text{FORWARD}(\mathbf{X}_{\mathcal{B}})$  ▷ Eqs. (1)–(3)
7:      $\boldsymbol{\delta}^{(L)} \leftarrow (\hat{\mathbf{Y}}_{\mathcal{B}} - \mathbf{Y}_{\mathcal{B}})/|\mathcal{B}|$  ▷ Eq. (7)
8:     Retropropagar  $\boldsymbol{\delta}$  e obter  $\nabla_{\theta} L$  ▷ Eqs. (8)–(9)
9:      $\mathbf{v} \leftarrow \mu \mathbf{v} - \eta \nabla_{\theta} L$ ;  $\theta \leftarrow \theta + \mathbf{v}$  ▷ Eq. (10)
10:   end for
11:   Calcular  $L_{va}$  e acurácias de treino/validação; registrar histórico
12:   if  $L_{va} < L^* - 10^{-6}$  then
13:      $L^* \leftarrow L_{va}$ ;  $\theta^* \leftarrow \theta$ ;  $p \leftarrow 0$ 
14:   else
15:      $p \leftarrow p + 1$ ; se  $p \geq P$  então interromper
16:   end if
17: end for
18: return  $\theta^*$ 
```

4.4 Verificação do gradiente

Antes do treinamento, o gradiente analítico foi comparado ao numérico, obtido por diferenças centrais ($\epsilon = 10^{-5}$) em todos os parâmetros de uma rede 4–5–4–3. A rede tem duas camadas ocultas para exercitar a recursão da Eq. (9). O critério de aceitação é $|g_a - g_n| / \max(|g_a| + |g_n|, 10^{-12}) < 10^{-6}$.

4.5 Experimentos e baselines

Em torno da configuração base, variou-se um fator por vez, com avaliação por k -fold ($k = 5$): número de neurônios ocultos $\{2, 4, 8, 16, 32, (10, 6)\}$, taxa de aprendizado $\{0,005; 0,01; 0,05; 0,1; 0,5\}$ e ativação {sigmoide, tanh, ReLU}. Como baselines, usaram-se a regressão logística multinomial, equivalente a uma rede Softmax sem camada oculta e, portanto, uma medida direta do ganho trazido pela não linearidade, e o k -NN com $k = 5$ (DASARATHY, 1980; GATES, 1972), ambos do scikit-learn, sobre os mesmos splits e partições. Treinou-se ainda uma MLP auxiliar apenas com os atributos de pétala, para visualizar a fronteira de decisão em duas dimensões.

4.6 Métricas

Para cada classe c : precisão $P_c = TP_c / (TP_c + FP_c)$, revocação $R_c = TP_c / (TP_c + FN_c)$ e $F1_c = 2P_cR_c / (P_c + R_c)$, além das médias *macro*, da acurácia global e das matrizes de confusão.

5 Resultados

Todos os valores desta seção foram gerados por `iris_mlp.py` (registrados em `resultados.json`). O notebook `iris_mlp.ipynb` reproduz os mesmos números.

5.1 Verificação do gradiente

O erro relativo máximo entre gradiente analítico e numérico foi $3,0 \times 10^{-8}$ (sigmoide), $3,5 \times 10^{-8}$ (*tanh*) e $1,5 \times 10^{-9}$ (ReLU), todos abaixo do limiar de 10^{-6} . A implementação da retropropagação, incluindo a Eq. (7), está correta.

5.2 Treinamento

A Figura 4 mostra as curvas de aprendizado. A parada antecipada selecionou a época 99, de um total de 199 épocas executadas. Nessa época, a entropia cruzada foi de 0,0524 no treino e 0,0402 na validação, com acurácias de 98,10% e 100,00%, respectivamente.

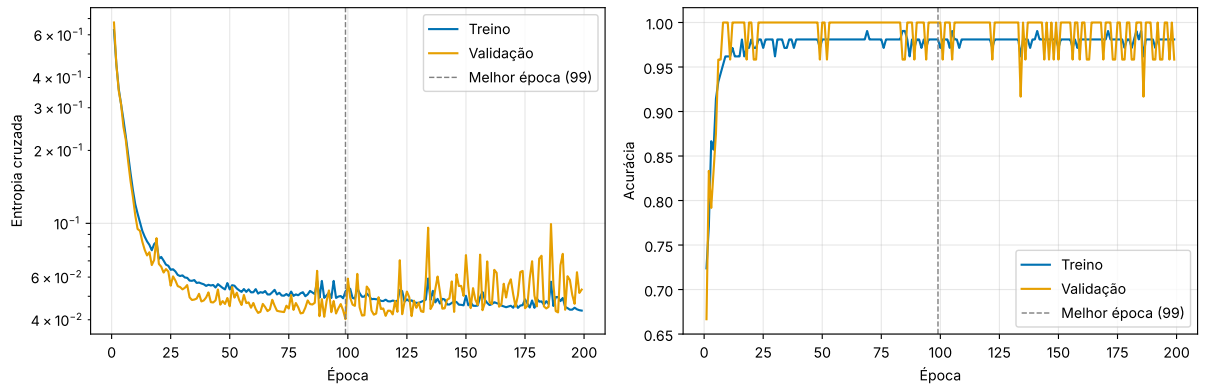


Figura 4: Curvas de perda (escala logarítmica) e de acurácia de treino e validação; a linha tracejada indica a época restaurada pela parada antecipada.

5.3 Desempenho no conjunto de teste

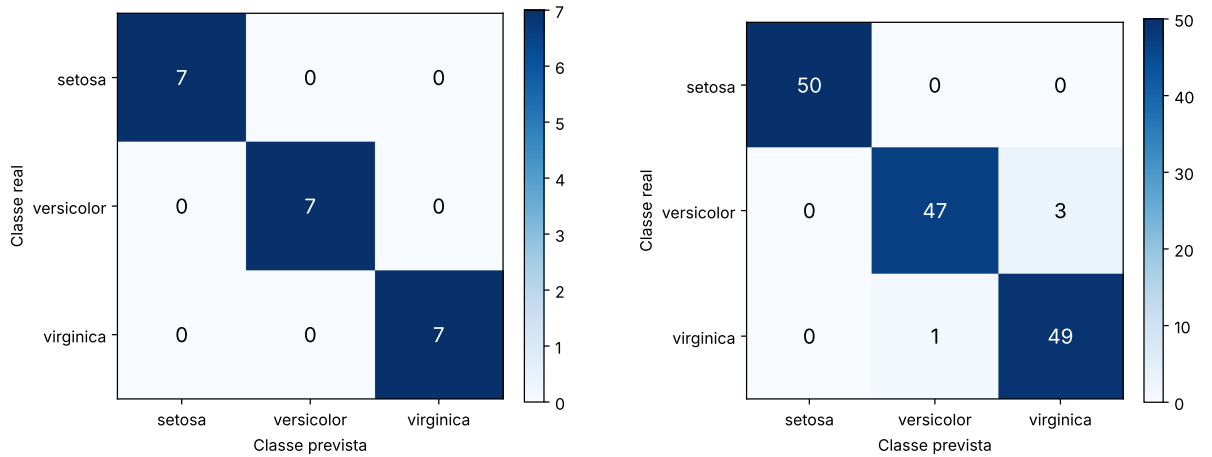
No conjunto de teste (21 amostras), a MLP obteve acurácia de 100,00%, F1 macro de 100,00% e entropia cruzada de 0,0050. As Tabelas 3 e 4 e a Figura 5 detalham os resultados.

Tabela 3: Métricas por classe no conjunto de teste.

Classe	Precisão	Revocação	F1
<i>Iris setosa</i>	1,000	1,000	1,000
<i>Iris versicolor</i>	1,000	1,000	1,000
<i>Iris virginica</i>	1,000	1,000	1,000
Média macro	1,000	1,000	1,000

Tabela 4: Matrizes de confusão (linhas: classe real; colunas: classe prevista).

Real \ Prevista	Teste (holdout, $n = 21$)			k-fold agregado ($n = 150$)		
	Set.	Ver.	Vir.	Set.	Ver.	Vir.
Setosa	7	0	0	50	0	0
Versicolor	0	7	0	0	47	3
Virginica	0	0	7	0	1	49

**(a)** Teste (holdout).**(b)** k-fold agregado (predições fora da partição).**Figura 5:** Matrizes de confusão da MLP 4-8-3.

5.4 Validação cruzada e experimentos

Com $k = 5$, a configuração principal obteve acurácia de $97,33\% \pm 3,65\%$ e F1 macro de $97,32\% \pm 3,67\%$. As acurácias por partição foram 100,00%; 93,33%; 93,33%; 100,00%; 100,00%. A Tabela 5 e a Figura 6 apresentam o estudo de sensibilidade.

Tabela 5: Sensibilidade a hiperparâmetros (k-fold estratificado, $k = 5$; média $\pm$ desvio-padrão). Os demais hiperparâmetros seguem a Tabela 2.

Fator	Valor	Acurácia	F1 macro	Melhor época (média)
Neurônios ocultos	2	96,67% $\pm$ 2,36%	96,66%	607
Neurônios ocultos	4	98,00% $\pm$ 2,98%	98,00%	228
Neurônios ocultos	8	97,33% $\pm$ 3,65%	97,32%	523
Neurônios ocultos	16	98,00% $\pm$ 2,98%	98,00%	263
Neurônios ocultos	32	97,33% $\pm$ 2,79%	97,32%	261
Neurônios ocultos	10-6	96,67% $\pm$ 2,36%	96,66%	368
Taxa η	0,005	97,33% $\pm$ 3,65%	97,32%	907
Taxa η	0,01	97,33% $\pm$ 3,65%	97,32%	673
Taxa η	0,05	97,33% $\pm$ 3,65%	97,32%	523
Taxa η	0,1	98,00% $\pm$ 2,98%	97,98%	500
Taxa η	0,5	95,33% $\pm$ 1,83%	95,31%	381
Ativação	sigmoid	97,33% $\pm$ 3,65%	97,32%	654
Ativação	tanh	97,33% $\pm$ 3,65%	97,32%	523
Ativação	relu	96,67% $\pm$ 3,33%	96,66%	485

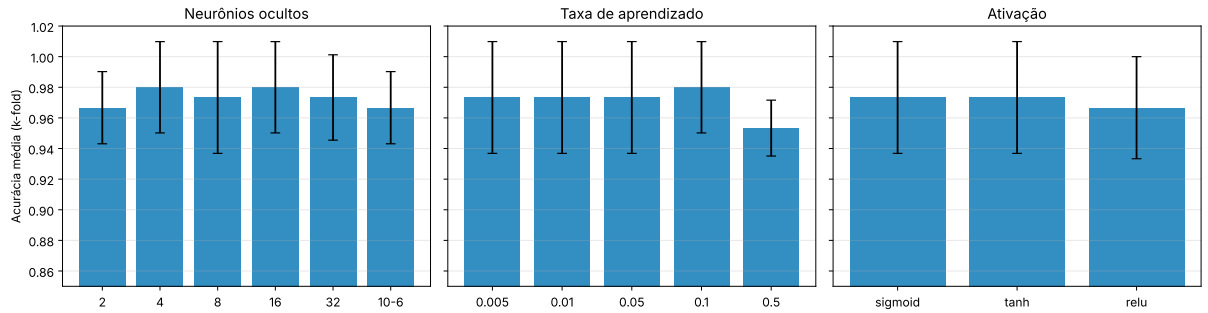


Figura 6: Acurácia média do k-fold ($\pm$ desvio-padrão) para cada fator variado.

5.5 Comparação com baselines

Tabela 6: Comparação com baselines (mesmo split holdout e mesmas partições do k-fold).

Modelo	Acurácia (teste)	Acurácia k-fold
MLP 4–8–3 (<i>tanh</i> , Softmax + CE)	100,00%	97,33% $\pm$ 3,65%
Regressão logística multinomial	100,00%	96,67% $\pm$ 3,33%
k -NN ($k = 5$)	100,00%	94,67% $\pm$ 2,98%

5.6 Fronteira de decisão

A MLP auxiliar treinada apenas com *petal length* e *petal width* obteve acurácia de teste de 100,00%. Suas regiões de decisão aparecem na Figura 7.

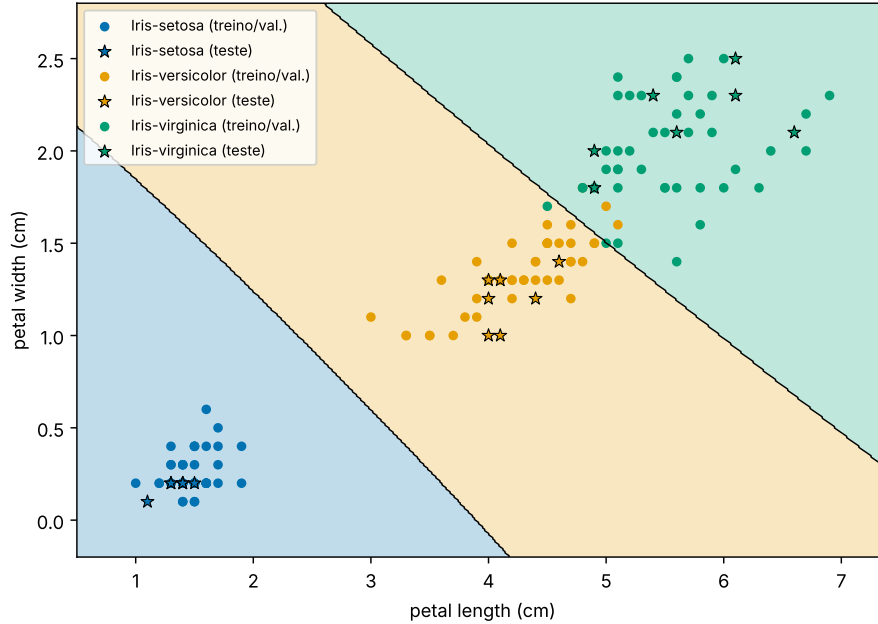


Figura 7: Regiões de decisão da MLP no plano das pétalas; as estrelas indicam as amostras de teste.

6 Discussão

Correção da implementação. O *gradient checking* confirma que a retropropagação manual, incluindo a simplificação $\partial L / \partial \mathbf{z} = \hat{\mathbf{y}} - \mathbf{y}$, está correta para as três ativações. Esse teste é o que dá confiabilidade a uma implementação feita do zero: sem ele, erros sutis de sinal ou de dimensão podem passar despercebidos, porque a rede às vezes "aprende" mesmo com gradientes incorretos.

Dinâmica de treinamento. As curvas da Figura 4 mostram queda rápida da perda nas primeiras épocas, seguida de estabilização. A perda de validação passa a oscilar em torno do mínimo, efeito do ruído do mini-batch e do tamanho reduzido do conjunto de validação. A parada antecipada restaura os pesos de menor perda de validação, o que funciona como regularização implícita.

Holdout versus k-fold. Com 7 amostras por classe no teste, a acurácia holdout (100,00%) tem alta variância e não deve ser lida isoladamente. A estimativa de referência é a do k-fold ($97,33\% \pm 3,65\%$), cujo desvio-padrão quantifica a variabilidade entre partições.

Confusão *Versicolor* $\times$ *Virginica*. A matriz agregada do k-fold (Tabela 4) mostra que a *Setosa* não é confundida com nenhuma outra classe, o que é coerente com

sua separabilidade linear. Dos 4 erros fora da partição, 4 ocorrem entre *Versicolor* e *Virginica*. A Figura 7 mostra a causa: as duas espécies se sobrepõem na região de pétalas intermediárias (comprimento de cerca de 4,5 a 5,1 cm e largura de 1,5 a 1,8 cm). Nessa região, amostras de classes distintas têm medidas praticamente iguais, e o erro é essencialmente irreduzível com esses quatro atributos.

Sensibilidade a hiperparâmetros. Na faixa avaliada, as variações de arquitetura, taxa de aprendizado e ativação produzem diferenças de desempenho da ordem do desvio-padrão entre partições (Tabela 5), ou seja, sem diferença estatisticamente relevante. Taxas menores exigem mais épocas até o mínimo de validação, e taxas muito altas tornam a otimização mais ruidosa. Redes com 2 neurônios já são suficientes, o que confirma que a fronteira necessária é simples. A escolha de 8 neurônios com *tanh* é, portanto, adequada, mas não crítica.

MLP versus baselines. A regressão logística multinomial, que é uma rede Softmax sem camada oculta, atinge desempenho próximo ao da MLP (Tabela 6), indicando que o problema é quase linearmente separável. O ganho possível da camada oculta limita-se às poucas amostras na região de sobreposição *Versicolor/Virginica*. O *k*-NN também apresenta desempenho elevado, coerente com os resultados históricos para o Iris (DASARATHY, 1980; GATES, 1972).

Limitações. A base é pequena (150 amostras), o que limita o poder estatístico das comparações entre configurações. Os experimentos variam um fator por vez e não exploram interações. Por fim, as diferenças observadas entre modelos são menores que a variabilidade entre partições.

7 Conclusão

Foi implementada do zero, em NumPy, uma MLP para o reconhecimento das três espécies de flores Iris, atendendo a todos os requisitos do enunciado: três classes balanceadas, quatro atributos em cm, ativação Softmax (em forma numericamente estável) na saída e função de custo entropia cruzada. A derivação do gradiente conjunto Softmax + entropia cruzada foi apresentada e validada numericamente. O protocolo experimental empregado incluiu z-score sem vazamento de dados, divisão estratificada, parada antecipada, k-fold estratificado, estudo de sensibilidade e baselines.

A rede 4–8–3 obteve 100,00% de acurácia no teste e $97,33\% \pm 3,65\%$ no k-fold, com reconhecimento perfeito da *Setosa* e erros restritos ao par *Versicolor–Virginica*, conforme previsto pela análise exploratória. Como trabalhos futuros, sugerem-se testes estatísticos pareados entre modelos (por exemplo, McNemar ou k-fold repetido), regularização L2 e estimativa de incerteza das probabilidades Softmax (calibração).

Referências

- DASARATHY, B. V. Nosing around the neighborhood: a new system structure and classification rule for recognition in partially exposed environments. **IEEE Transactions on Pattern Analysis and Machine Intelligence**, v. PAMI-2, n. 1, p. 67–71, 1980.
- DUDA, R. O.; HART, P. E. **Pattern classification and scene analysis**. New York: John Wiley & Sons, 1973.
- FISHER, R. A. The use of multiple measurements in taxonomic problems. **Annals of Eugenics**, v. 7, n. 2, p. 179–188, 1936.
- FISHER, R. A. **Iris**. UCI Machine Learning Repository, 1988. Disponível em: <http://archive.ics.uci.edu/ml/machine-learning-databases/iris/>. Acesso em: 7 out. 2026.
- GATES, G. W. The reduced nearest neighbor rule. **IEEE Transactions on Information Theory**, v. 18, n. 3, p. 431–433, 1972.
- GLOROT, X.; BENGIO, Y. Understanding the difficulty of training deep feedforward neural networks. In: INTERNATIONAL CONFERENCE ON ARTIFICIAL INTELLIGENCE AND STATISTICS, 13., 2010, Sardinia. **Proceedings [...]**. PMLR, 2010. p. 249–256.
- GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. **Deep learning**. Cambridge, MA: MIT Press, 2016.
- HAYKIN, S. **Redes neurais: princípios e prática**. 2. ed. Porto Alegre: Bookman, 2001.
- HE, K.; ZHANG, X.; REN, S.; SUN, J. Delving deep into rectifiers: surpassing human-level performance on ImageNet classification. In: IEEE INTERNATIONAL CONFERENCE ON COMPUTER VISION, 2015, Santiago. **Proceedings [...]**. IEEE, 2015. p. 1026–1034.
- RUMELHART, D. E.; HINTON, G. E.; WILLIAMS, R. J. Learning representations by back-propagating errors. **Nature**, v. 323, p. 533–536, 1986.

A Trechos do código

Os trechos a seguir foram extraídos de `iris_mlp.py` (as docstrings foram omitidas).

```
1 def softmax(z: np.ndarray) -> np.ndarray:
2     z_desl = z - np.max(z, axis=1, keepdims=True)
3     e = np.exp(z_desl)
4     return e / np.sum(e, axis=1, keepdims=True)
```

Código 1: Softmax numericamente estável (Eq. 5).

```
1 def cross_entropy(y: np.ndarray, y_hat: np.ndarray, eps: float = 1e-12) -> float:
2     return float(-np.mean(np.sum(y * np.log(np.clip(y_hat, eps, 1.0)), axis=1)))
```

Código 2: Entropia cruzada categorial (Eq. 6).

```
1 def forward(self, X: np.ndarray) -> np.ndarray:
2     self._A = [X]
3     self._Z = []
4     a = X
5     for l in range(len(self.W)):
6         z = a @ self.W[l] + self.b[l] # Eq. (1)
7         self._Z.append(z)
8         a = softmax(z) if l == len(self.W) - 1 else self.f(z)
9         self._A.append(a)
10    return a
11
12 def backward(self, Y: np.ndarray):
13     N = Y.shape[0]
14     delta = (self._A[-1] - Y) / N # Softmax + CE
15     dW = [np.empty(0)] * len(self.W)
16     db = [np.empty(0)] * len(self.W)
17     for l in range(len(self.W) - 1, -1, -1):
18         dW[l] = self._A[l].T @ delta # gradiente dos pesos
19         db[l] = delta.sum(axis=0)
20         if l > 0:
21             delta = (delta @ self.W[l].T) * self.df(self._Z[l - 1])
22    return dW, db
23
24 def update(self, dW, db, lr: float, momentum: float = 0.0) -> None:
25     for l in range(len(self.W)):
26         self.vW[l] = momentum * self.vW[l] - lr * dW[l]
27         self.vb[l] = momentum * self.vb[l] - lr * db[l]
28         self.W[l] += self.vW[l]
29         self.b[l] += self.vb[l]
```

Código 3: Propagação direta, retropropagação e atualização com momentum (métodos da classe MLP).